

R4.A.L1 – Introduction à l'Intelligence Artificielle, partie 2

TD n°4 : Prolog, compléments

Exercice 1. Reprendre l'exercice 3 du TD 3 : les faits de base définissant l'arbre généalogique de la famille considérée étaient

homme(albert).	femme(marie).	pere(louis,benoit).	mere(marie,sophie).
homme(jean).	femme(germaine).	pere(albert,jean).	mere(marie,bertrand).
homme(paul).	femme(sophie).	pere(jean,simone).	mere(germaine,jean).
homme(bertrand).	femme(christiane).	pere(paul,sophie).	mere(christiane,simone).
homme(louis).	femme(simone).	pere(jean,paul).	mere(christiane,paul).
homme(benoit).		pere(paul,bertrand).	mere(simone,benoit).

L'expression de cette base de faits n'est pas optimale, certains informations sont redondantes. En effet, puisqu'on dispose des prédicats `homme(X)` et `femme(X)`, on peut se contenter du prédicat `parent_de(X,Y)` pour pouvoir déduire la définition de `pere(X,Y)` et `mere(X,Y)`...

1. Réécrivez l'ensemble de faits ci-dessus en n'utilisant que les prédicats `homme(X)`, `femme(X)` et `parent(X)`, pour redéfinir à l'identique l'arbre généalogique de la famille.
2. Définissez les prédicats suivants :
 - (a) `pere(X,Y)` : X est le père de Y ;
 - (b) `mere(X,Y)` : X est la mère de Y ;
 - (c) `grand_parent_de(X,Y)` : X est un grand parent de Y ;
 - (d) `avoir_un_enfant(X)` : X a au moins un enfant ;
 - (e) `etre_un_enfant(X)` : X est l'enfant de quelqu'un ;
 - (f) `oncle(X,Y)` : X est un oncle de Y ;
 - (g) `tante(X,Y)` : X est une tante de Y ;
 - (h) `cousin(X,Y)` : X est un cousin de Y ;
 - (i) `cousine(X,Y)` : X est une cousine de Y ;
 - (j) `ancetre(X,Y)` : X est un ancêtre de Y (*Suggestion* : Utilisez la récursivité ; X est un ancêtre de Y signifie que, ou bien X est un parent de Y, ou bien X est un parent d'un ancêtre de Y) ;
 - (k) `marie(X,Y)` : X et Y sont mariés (on suppose que pour être mariés, il suffit d'avoir un enfant en commun).

Quelles sont les tantes, cousines, cousins, et ancêtres de cette famille ?

Exercice 2. Supposons qu'on dispose de la seule base de faits

`rond(sphere).`

sans aucune règle. Si on pose au moteur Prolog la question

?- `rond(terre).`

la réponse obtenue est `false`, tout simplement parce que Prolog ne connaît pas le fait `rond(terre)`, et ne peut le déduire à partir du seul fait `rond(sphere)` connu. Par contre, à partir du prédicat disponible pour la négation, `\+`, la réponse à la question

?- \+(rond(terre)).

sera **true**. En effet, \+, défini en tant que *négarion par défaut* permet de dériver que *tout ce qui n'est pas déclaré comme fait ou n'est pas déductible de la base courante est réputé faux*. Autrement dit, tout ce qui ne peut pas être dérivé est faux (forme de négation différente de la *négarion logique*). A partir de ce constat,

1. Ecrivez la règle définissant le prédicat **nonrond(X)** comme la négation par défaut du prédicat **rond**, sans utiliser \+ mais en utilisant le cut !, les prédicats 0-aire **fail** et **true**, et la *variable anonyme* (« any » : _).
2. Définissez un prédicat **neg(X)** pour la négation : si **X** est vrai, alors **neg(X)** renvoie **false**; si **X** est faux, alors **neg(X)** renvoie **true**.
3. Vérifiez que **neg(rond(X))** se comporte de la même manière que **nonrond(X)** pour plusieurs valeurs de la variable **X** (par exemple, **sphere** et **terre**).

Exercice 3. Prolog introduit la notion de *liste* comme une succession d'éléments séparés par des virgules et entourée de crochets, par exemple :

```
[alain, alexis, frederic, safa, vincent]
[1, alain, alexis, toto(X), Z, frederic, safa, vincent]
[alain, [alexis, [frederic, [safa, [vincent]]]]]]
[]
```

On considère qu'une liste est toujours constituée d'une tête (premier élément) et d'une queue (la sous-liste restante); par exemple, la tête et la queue de la liste

```
[alain, alexis, frederic, safa, vincent]
```

sont respectivement **alain** et **[alexis, frederic, safa, vincent]**. Afin de décomposer commodément une liste en tête et queue, Prolog met à disposition le séparateur | (pipe); par exemple :

```
[X|Y] = [alain, alexis, frederic, safa, vincent]
```

renvoie :

```
X = alain
Y = [alexis, frederic, safa, vincent]
```

Explorer une liste pour savoir si un élément est présent dans une liste se fait récursivement en deux lignes de programme :

```
element_de(X,[X,Q]).
element_de(X,[T,Q]) :- element_de(X,Q).
```

Expérimenter ces deux lignes sur l'une des listes données en exemple, puis en vous inspirant de la façon dont elles sont écrites,

1. proposer un nouveau prédicat **elements_impairs_de(L,LI)** permettant d'extraire tous les éléments de rang impairs de la liste **L** dans la liste **LI** et le tester sur la liste **L = [0, 1, 2, 3, 4, 5]**;
2. proposer un prédicat permettant de produire la liste inversée d'une liste initiale.

Exercice 4. Donner les règles qui définissent le prédicat **hors_de(X,L)**, qui vérifie que l'objet **X** n'appartient pas à la liste **L** (*Suggestion* : utiliser le prédicat prédéfini **dif(X,Y)**).

Exercice 5. Donner les règles qui définissent le prédicat **différents(L)**, destiné à vérifier que la liste **L** ne contient pas de répétition (*Suggestion* : utiliser le prédicat **hors_de(X,L)** défini à l'exercice précédent).

Exercice 6 (Les tours de Hanoï). *Les tours de Hanoï* constituent un jeu de réflexion imaginé par le mathématicien français Edouard Lucas, et consistant à déplacer des disques de diamètres différents d'une tour de « départ » à une tour d'« arrivée » en passant par une tour « intermédiaire », ceci en un nombre minimum de coups, tout en respectant les règles suivantes :

- on ne peut déplacer plus d'un disque à la fois ;
- on ne peut placer un disque que sur un autre disque plus grand que lui ou sur un emplacement vide.

On suppose que cette dernière règle est également respectée dans la configuration de départ. Ecrire les règles permettant de calculer la suite des mouvements conduisant à une solution pour une tour comportant N disques (*Suggestion* : Définir le prédicat `mouvement(N,Start,Interm,End,S)` qui, étant donnés N le nombre de disques, **Start** la tour de départ, **Interm** la tour intermédiaire, **End** la tour d'arrivée, produit **S** la liste des mouvements élémentaires composant une solution ; utiliser le prédicat prédéfini `append(L1,L2,L3)` qui permet d'obtenir dans la liste **L3** la concaténation des listes **L1** et **L2**).

Exercice 7 (bonus). Cinq maisons, de cinq couleurs différentes, sont alignées le long d'une route. Dans chacune de ces maisons vit une personne de nationalité différente. Chacune de ces personnes boit une boisson différente, fume une marque de cigare différente et a un animal favori différent. On sait que

- Le Britannique vit dans la maison rouge.
- Le chien appartient à l'Espagnol.
- On boit du café dans la maison verte.
- l'Ukrainien boit du thé.
- La maison verte est à côté de la maison blanche, à droite.
- Le fumeur de Old Glod élève des escargots.
- On fume des Kool dans la maison jaune.
- On boit du lait dans la maison du milieu.
- Le Norvégien habite la première maison à gauche.
- Le fumeur de Chesterfield habite à côté du propriétaire du renard.
- Le fumeur de Kool habite à côté du propriétaire du cheval.
- Le fumeur de Gitanes boit du vin.
- Le japonais fume des Craven.
- Le Norvégien habite à côté de la maison bleue.

Qui boit de l'eau ? A qui appartient le zèbre ? (*Indications* : On supposera que les maisons sont numérotées de un à cinq, et on exprimera les relations de voisinage au moyen de la liste des cinq premiers entiers. De même, on définira une liste de couleurs, de boissons, d'animaux, de marques de cigarettes. On exprimera que toutes ces listes comportent cinq éléments distincts, et que ces éléments sont liés par les relations énoncées. Utiliser le prédicat `différents(L)` défini à l'exercice 5).